Introduction

Research Tools as Epistemic Infrastructure I

Research Tools as Epistemic Infrastructure II

Scientific research operates through a layered ecology of tools, documents, and practices—each shaping what can be known, communicated, and verified. When these layers are fragmented, the artifacts of research (manuscripts, data, code, figures) drift out of alignment with one another, creating gaps that are structural rather than incidental. The "reproducibility crisis" is one symptom of this deeper misalignment: a 2016 *Nature* survey of 1,576 researchers found that 70% had tried and failed to reproduce another scientist's experiments, and more than half had failed to reproduce their own [@baker2016reproducibility]. Freedman et al. estimate that the biomedical industry alone loses $28 billion annually to irreproducible preclinical research [@freedman2015economics]. But reproducibility is only the most visible face of a broader problem. Research software engineering, epistemic integrity, and the coordination between human researchers and AI collaborators all depend on the same underlying question: whether the tools that produce research artifacts can themselves be made transparent, testable, and self-documenting. Nosek et al. [@nosek2018preregistration] have argued that the preregistration

Related Work I

Gentleman and Temple Lang [@gentleman2007research] introduced the concept of a *research compendium*—a single unit of scholarly communication bundling code, data, and narrative. This vision has driven two decades of tooling, which can be grouped into four categories: workflow managers, literate programming systems, containerization approaches, and best-practice frameworks.

Related Work II

Workflow Managers

Snakemake [@koster2012snakemake] uses a rule-based, Python-derived DSL to specify computational workflows as directed acyclic graphs of file-producing steps. It supports containerized execution via Conda and Singularity environments. Snakemake 9.x (2024–2025) introduced a plugin architecture for extended execution backends and storage providers, yet its scope remains computational pipeline orchestration—it does not integrate manuscript rendering, testing enforcement, or provenance watermarking.

Nextflow [@ditommaso2017nextflow] employs a dataflow programming paradigm with native support for container-based execution across heterogeneous computing environments (local, SLURM, AWS). Like Snakemake, Nextflow excels at bioinformatics pipeline parallelism but does not address manuscript production, document integrity, or the testing–publication coupling that characterizes research reproducibility.

CWL (Common Workflow Language) [@amstutz2016cwl] provides

`template/`: An Integrated Solution I

`template/` was conceived as a structural antidote to this fragmentation. Rather than adding reproducibility as an afterthought—a Docker container wrapping an already-disjointed workflow [@boettiger2015docker]—the template enforces integrity at the architectural level. It realizes Gentleman and Temple Lang's research compendium vision [@gentleman2007research] at repository scale, bundling code, data, tests, manuscripts, and provenance into a single, pipeline-enforced system with version-controlled infrastructure [@ram2013git]. It stands on four primary pillars:

1. **Ergonomic Modularity**: A Two-Layer Architecture cleanly separates globally shared infrastructure (logging, rendering, validation, steganography) from project-specific logic (manuscripts, scripts, data). 22 infrastructure subpackages comprising ~456 Python modules provide reusable services; projects consume them without modification.

`template/`: An Integrated Solution II

2. **Execution Integrity**: A Zero-Mock testing policy where pipeline advancement is contingent on test passage. Infrastructure tests must achieve 60% coverage; project tests must achieve 90%. All tests use real filesystem operations, real subprocess calls, and real network connections—no mock objects, no fake services, no synthetic test doubles. ~7,024 infrastructure tests and 646+ project tests enforce this standard.

3. **Automated Provenance**: Steganographic watermarking and cryptographic hashing are integrated directly into the rendering pipeline. Every generated PDF carries a SHA-256 fingerprint, an alpha-channel text overlay encoding the build timestamp and commit hash, and optionally a QR code linking to the repository. Provenance is not asserted by policy; it is enforced by architecture.

4. **AI-Agent Collaboration and Skill-Based Agentic Operations**: A three-tier documentation architecture enables AI agents to operate at every level of the system. At the system level, `CLAUDE.md` asserts global architectural constraints. At the structural level, `AGENTS.md` files at every directory expose local API surfaces, file inventories, and integration contracts. At the module level, `SKILL.md` files—written to a discoverable YAML+Markdown schema aligned with the Model Context Protocol [@anthropic2024mcp]—define each infrastructure module as a reusable, self-describing tool. An agent invoking `infrastructure.rendering` does not need to read source code: it reads the `rendering/SKILL.md`, which declares the module's name, description, key imports, and example invocations in a machine-parseable YAML frontmatter block. This architecture is the practical realization of the skill-library paradigm established in the agent literature: Yao et al.'s ReAct framework [@yao2023react] demonstrated that interleaving reasoning traces with tool invocations dramatically improves

Scope and Contributions I

This paper is itself a product of the template it describes. The metrics populating its tables were computed by the introspection module documented in the Methods; the figures were rendered by the visualization code validated by the test suite described in Quality Assurance; the PDF carrying these words was assembled by the same YAML-declared pipeline whose architecture is the subject of the Results. This self-productive loop—where the system that is described is also the system that produces the description—is not incidental but structural, a concrete demonstration that `template/` can sustain the full lifecycle from source code to published artifact within a single, version-controlled, pipeline-enforced repository. Our contributions are:

- A formal description of the Two-Layer Architecture and Standalone Project Paradigm that enables N independent research projects to share infrastructure without coupling.

Scope and Contributions II

- A detailed specification of the 12-stage DAG in `infrastructure/core/pipeline/pipeline.yaml`: default full runs use 10 stages; `--core-only` runs 8; opt-in bundle and archival stages via `--tags`.
- A comparative analysis positioning `template/` against ten peer tools—Snakemake, Nextflow, CWL, Quarto, Jupyter Book, R Markdown, DVC, Typst, Overleaf, OpenAI Prism—across fourteen feature dimensions, demonstrating that `template/` uniquely bundles the fourteen enforcement capabilities enumerated in §Results.
- An empirical evaluation across the three canonical exemplar projects under `projects/` (`template_active_inference`, `template_autoresearch_project`, `template_code_project`, `template_prose_project`, `template_template`)—plus this meta manuscript from `projects_in_progress/template`, which exercises introspection-derived metrics.

Scope and Contributions III

- A security analysis of the steganographic provenance layer, including a formal threat model and tamper-detection capabilities aligned with the W3C PROV data model [@morea u2013provdm] and SLSA [@openssf2023slsa].
- An open-source reference implementation available at `github.com/docxology/template`.

Paper Organization I

The Methods describe the Two-Layer Architecture, Thin Orchestrator pattern, pipeline stages, and AI collaboration model. Results present quantitative metrics from multi-project execution, coverage analysis, and steganographic benchmarks. The Discussion addresses the Zero-Mock tradeoff, scalability implications, a detailed tool comparison, and future directions. The Infrastructure Module Reference provides detailed documentation for all 22 subpackages. Security and Provenance describes the steganographic and cryptographic integrity layer. The Appendices provide pipeline, configuration, and comparative references.